Συστήματα Υπολογισμού Υψηλών Επιδόσεων
Project 4

Τσόγκας Παναγιώτης	**Νίκας Ιωάννης Ιάσων**	**Τσιαντός Δημήτριος**
ptsogkas@uth.gr	ioannikas@uth.gr	dtsiantos@uth.gr
3672	3771	3796

Δεκέμβριος 2025

Περιεχόμενα

1 **Ο αλγόριθμος** 3

2 **Αρχική υλοποίηση** 4

3 **Υλοποίηση με δύο kernels** 4

4 **Εξάληψη ασσυμετρίας του kernel** 4

5 **Αποφυγή Atomic serialization με χρήση privitization** 5

6 **Υλοποίηση Inclusive parallel scan** 6

7 **Memory Coalescence** 6

8 **Χρήση streams** 7

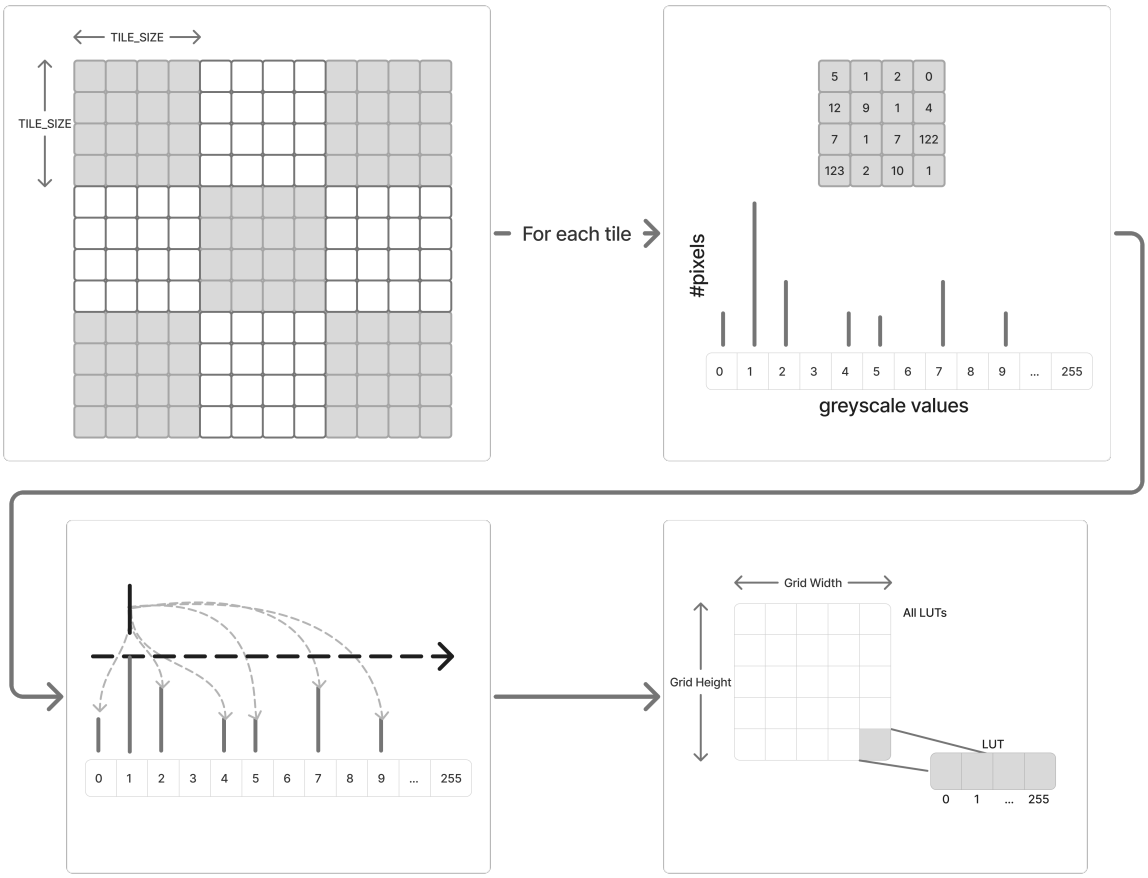
9 **Χρήση events** 8

10 **Δεύτερο kernel** 10

11 **Χρόνοι εκτέλεσης** 10

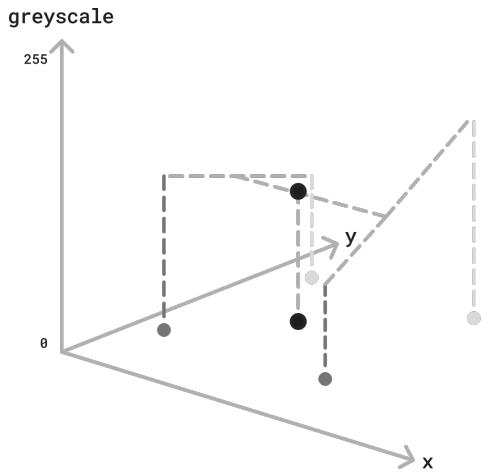
1 Ο αλγόριθμος

Ο αλγόριθμος CLAHE μπορεί να χωριστεί σε δύο μεγάλα βήματα. Στο πρώτο βήμα χωρίζουμε την εικόνα σε tiles και δημιουργούμε ένα ιστόγραμμα για κάθε tile με τα greyscale values από $0 - 255$ στον οριζόντιο άξονα και το πλήθος των pixels στον κατακόρυφο. Στην συνέχεια αφαιρούμε από το ιστόγραμμα το πλήθος των τιμών που ξεπερνούν ένα προκαθορισμένο όριο και αναδιανέμουμε την ποσότητα αυτή ισόποσα στις υπόλοιπες στήλες. Τέλος, υπολογίζουμε και αποθηκεύουμε για κάθε tile το CDF του ιστογράμματος σε έναν πίνακα ο οποίος είναι και το αποτέλεσμα του πρώτου βήματος.



Σχήμα 1: Πρώτο βήμα αλγορίθμου.

Στο δεύτερο βήμα εφαρμόζουμε για κάθε pixel bilinear interpolation, ώστε η τελική τιμή έντασης να προκύψει από τα τέσσερα γειτονικά tiles. Για κάθε pixel εντοπίζουμε τα δύο κοντινότερα tiles στον οριζόντιο άξονα και τα δύο στον κατακόρυφο. Από τον πίνακα του πρώτου βήματος προσπελαύνουμε τις αντίστοιχες τιμές του CDF για το greyscale value του συγκεκριμένου pixel. Με αυτόν τον τρόπο εξασφαλίζεται ότι η μετάβαση ανάμεσα στα tiles γίνεται σταδιακά.



Σχήμα 2: Δεύτερο βήμα αλγορίθμου. Παράδειγμα bilinear interpolation.

2 Αρχική υλοποίηση

Στην πιο απλή υλοποίηση του αλγορίθμου, δημιουργήσαμε τρία διαφορετικά kernels. Ο πρώτος υπολογίζει το histogram και το αποθηκεύει στην global memory ώστε να το χρησιμοποιήσει ο δεύτερος kernel που υπολογίζει τα LUTs για κάθε tile. Έπειτα ο τρίτος kernel χρησιμοποιεί τα LUTs ώστε να υπολογίσει τις τελικές τιμές των pixel πραγματοποιώντας bilinear interpolation. Αυτό μας επιτρέπει να χρησιμοποιήσουμε το finest granularity στα threads που δημιουργούμε, δηλαδή χρησιμποιώντας ένα thread ανά pixel στον πρώτο και τρίτο kernel και ένα thread ανά ένα κελί του histogram στον δεύτερο kernel. Αντιθέτως όμως αυξάνει τις μεταφορές μεταξύ των global και shared memory. Οπότε προχωρήσαμε στην αλλαγή της δομής των kernel ώστε να μειωθούν τα transfers αλλά και για να επιτρέψουμε πιο έυκολες βελτιστοποιήσεις σε επόμενες εκδόσεις.

3 Υλοποίηση με δύο kernels

Η υλοποίηση η οποία βελτιώνει την προηγούμενη χρησιμοποιεί 2 kernels όπου το πρώτο υπολογίζει το histogram, χρησιμοποιώντας ένα thread ανά pixel, και ένα block ανά tile ενώ τα πρώτα 256 threads από κάθε block υπολογίζουν και τις τιμές για το LUT του συγκεκριμένου tile. Έτσι το histogram δεν χρειάζεται πλέον να μεταφέρεται καθόλου στην global εφόσον μετά τον υπολογισμό των LUTs δεν ξαναχρησιμοποιείται. Όμως έτσι δεσμεύουμε πόρους οι οποίοι δεν χρησιμοποιούνται για όλη την διάρκεια εκτέλεσης του kernel. Για αυτό προχωρήσαμε σε μια ενδιάμεση λύση που περιγράφεται παρακάτω.

Run	Average Throughput (MPixels/s)	Std. Deviation	Speed-up vs Previous
3-kernels	703.491	21.390	-
2-kernels	470.069	5.089	0.668

Table 1: Speed comparison: 3-kernels → 2-kernels

4 Εξάληψη ασσυμετρίας του kernel

Με τη συγχώνευση του κώδικα σε δύο kernels, όπως είδαμε στο προηγούμενο βήμα, προέκυψε μια ασυμμετρία στον τρόπο που γίνονται οι κλήσεις των kernels. Συγκεκριμένα, ζητάμε από τον Streaming Multiprocessor να χρονοδρομολογήσειblocks με 1024 threads, όμως πριν ολοκληρωθεί η εκτέλεσή τους, απελευθερώνουμε μόνο ένα μέρος από αυτά, καθώς μόνο τα πρώτα 8 warps έχουν εναπομείναντα κώδικα να τρέξουν ενώ τα υπόλοιπα 24 τελειώνουν.

Αυτό το μοτίβο μπορεί να οδηγήσει σε κατάσταση όπου threadsRequested − threadsUsed = 768 threads παραμένουν ανενεργά, επειδή δεν υπάρχουν αρκετά ελεύθερα threads για να εκτελεστεί ένα επιπλέον block. Ως αποτέλεσμα, η εκτέλεση συνεχίζεται σειριακά, περιορίζοντας τον βαθμό παραλληλισμού και μειώνοντας την απόδοση.

Στη συγκεκριμένη υλοποίηση παρατηρούμε ότι έχουμε 100% thread usage κατά τη δημιουργία του histogram, αλλά μόλις 25% thread usage στο επόμενο στάδιο. Για τον λόγο αυτό, υποτετραπλασιάσαμε τον αριθμό των threads σε κάθε kernel block στα 256. Με αυτόν τον τρόπο, κάθε thread υπολογίζει πλέον τέσσερα στοιχεία του histogram, ενώ ένα thread υπολογίζει ένα index από τον αντίστοιχο πίνακα πιθανότητας LUT που αντιστοιχεί στο κάθε tile.

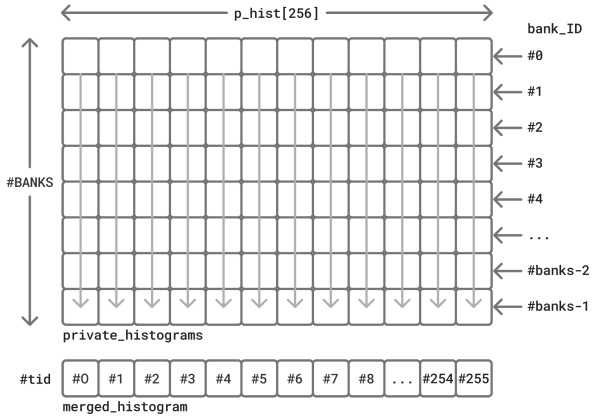
Με αυτή τη μέθοδο δεν παραμένει κανένα thread αχρησιμοποίητο, εξαλείφοντας έτσι το ενδεχόμενο bottleneck παραλληλοποίησης κατά τη χρονοδρομολόγηση από τον Streaming Multiprocessor, τουλάχιστον όσον αφορά τον αριθμό των διαθέσιμων threads.

Run	Average Throughput (MPixels/s)	Std. Deviation	Speed-up vs Previous
2-kernels	470.069	5.089	-
symmetric	722.906	14.800	1.538

Table 2: Speed comparison: 2-kernels → symmetric

5 Αποφυγή Atomic serialization με χρήση privitization

Ένα πρόβλημα που παρατηρούμε στις προηγούμενες υλοποιήσεις είναι η συνεχής χρήση `atomicAdd`, η οποία μπορεί να οδηγήσει σε atomic contention. Σε αυτή την κατάσταση, πολλαπλές κλήσεις της ίδιας συνάρτησης προς την ίδια διεύθυνση μνήμης αναγκάζονται να εκτελεστούν σειριακά, δημιουργώντας σημαντική καθυστέρηση. Για τον λόγο αυτό, στη συγκεκριμένη υλοποίηση χρησιμοποιήσαμε privatization.



Σχήμα 3: Δομή υλοποίησης privatization

Η δομή της υλοποίησης περιλαμβάνει έναν πίνακα `__shared__`, του οποίου κάθε γραμμή αντιστοιχεί σε ένα private copy του histogram. Κάθε private αντίγραφο χρησιμοποιείται από threads με διαδοχικά thread IDs (tid), κατανεμημένα κυκλικά ανά `NUM_BANKS`. Επειδή ο τρόπος προσπέλασης των θέσεων του histogram είναι σχεδόν τυχαίος, ο αριθμός των BANKS μειώνει ουσιαστικά την πιθανότητα serialized αιτημάτων προς τη μνήμη.

Run	Average Throughput (MPixels/s)	Std. Deviation	Speed-up vs Previous
symmetric	722.906	14.800	-
no-atomic-serial	854.920	18.813	1.183

Table 3: Speed comparison: symmetric → no-atomic-serial

6 Υλοποίηση Inclusive parallel scan

Για τον υπολογισμό του CDF σε κάθε tile στο kernel `compute_histogram` παρατηρούμε πως απαιτείται η χρήση του αλγορίθμου prefix sum. Μέχρι στιγμής η υλοποιήση του γίνοταν έχοντας κάθε thread να υπολογίζει το άθροισμα για τον εαυτό του το οποίο είναι πολλές παραπάνω πράξεις από ότι το ιδανικό. Για αυτό υλοποιήσαμε την έκδοση η οποία εκτελείται παράλληλα και εντός της shared memory. Συγκεκριμένα, ο αλγόριθμος λειτουργεί επαναληπτικά με λογαριθμικό αριθμό βημάτων. Σε κάθε επανάληψη, το βήμα (stride) διπλασιάζεται και κάθε νήμα αθροίζει την τιμή του στοιχείου που βρίσκεται σε απόσταση stride με τη δική του. Η διαδικασία αυτή απαιτεί προσεκτικό συγχρονισμό (__syncthreads()) για την αποφυγή race conditions κατά την ανάγνωση και εγγραφή στην κοινόχρηστη μνήμη. Όσον αφορά τα bank conflicts, η υλοποίηση τα αποφεύγει χάρη στη γραμμική προσπέλαση της μνήμης. Εντός ενός warp, τα διαδοχικά threads προσπελαύνουν διαδοχικές διευθύνσεις μνήμης (με σταθερή μετατόπιση stride). Το ουσιαστικό κέρδος είναι ότι έτσι αποφεύγουμε το serialization που θα προέκυπτε αν κάθε thread υπολόγιζε το δικό του CDF σειριακά.

Run	Average Throughput (MPixels/s)	Std. Deviation	Speed-up vs Previous
no-atomic-serial	854.920	18.813	-
scan-impl	897.553	32.925	1.050

Table 4: Speed comparison: no-atomic-serial → scan-impl

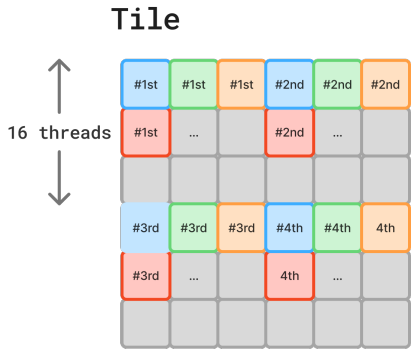
7 Memory Coalescence

Προτού προχωρήσουμε, είναι χρήσιμο να εξετάσουμε τα memory accesses που πραγματοποιούμε στο global memory και κατά πόσο αυτά είναι αποδεκτά.

Στον πρώτο kernel έχουμε τα ακόλουθα global memory accesses:

```
for (int cur_y = ty; cur_y < TILE_SIZE; cur_y += step_y) {
    for (int cur_x = tx; cur_x < TILE_SIZE; cur_x += step_x) {
        int gx = x_start + cur_x;
        int gy = y_start + cur_y;
        if (gx < w && gy < h) {
            unsigned char pix = data[gy * w + gx];
            ....
        }
    }
}
```

Στο παραπάνω τμήμα του κώδικα, προσπελαύνουμε τη μεταβλητή data στο global memory με τρόπο που επιτρέπει στα threads να εκτελούν παράλληλα και συνεχή αιτήματα προς τις αντίστοιχες θέσεις μνήμης, με αποτέλεσμα τα accesses να είναι coalesced. Στην Εικόνα 4 τα threads απεικονίζονται με διαφορετικά χρώματα και οι αριθμοί υποδεικνύουν τις σειριακές προσπελάσεις κάθε thread. Καθώς οι προσπελάσεις γίνονται σε ομάδες των 16 threads, επιτυγχάνεται coalescing.



Σχήμα 4: Access pattern εντός του tile.

Με παρόμοια λογική, στον πρώτο kernel κατά την εκτέλεση του:

```
// Write to Global Memory
unsigned char *lut = &(all_luts[(blockIdx.y * gridDim.x + blockIdx.x) * 256]);
```

επειδή πραγματοποιούνται 256 διαδοχικές και ταυτόχρονες προσπελάσεις, το access pattern είναι επίσης coalesced. Σχετικά με τον kernel `render_clahe` τα οι προσπελάσεις δεν είναι απαραίτητα coalesced καθώς το μοτίβο προσπέλασης εξαρτάται από το input. Βέβαια η διευθύνσεις βρίσκονται σε μικρό range (0-256) οπότε αυτό αυξάνει την πιθανότητες κάποιες προσπελάσεις να είναι coalesced.

```
val = img_in[y * w + x];

// Fetch mapped values from the 4 nearest tile LUTs
tl = all_luts[(y1 * gridDim.x + x1) * 256 + val];
tr = all_luts[(y1 * gridDim.x + x2) * 256 + val];
bl = all_luts[(y2 * gridDim.x + x1) * 256 + val];
br = all_luts[(y2 * gridDim.x + x2) * 256 + val];
...

img_out[y * w + x] = (unsigned char)(final_val + 0.5f);
```

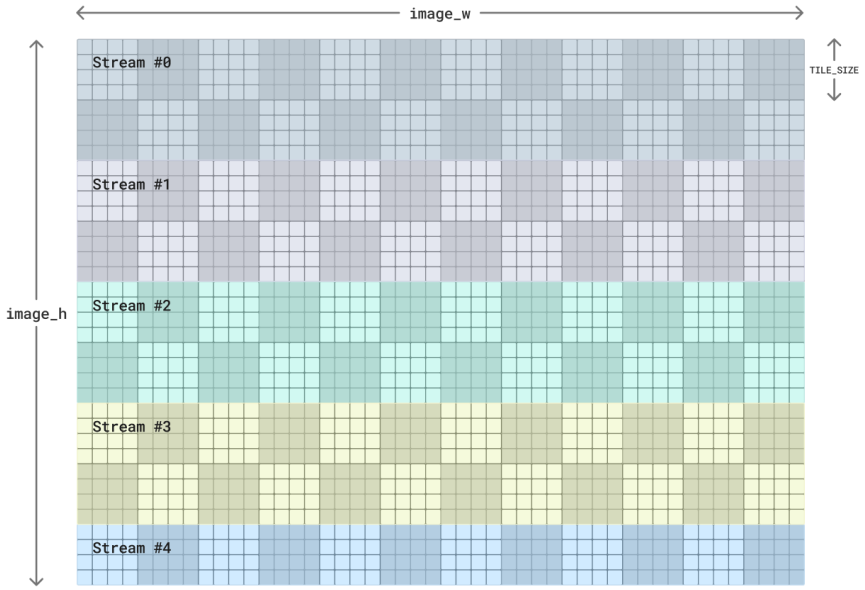
8 Χρήση streams

Παρατηρώντας τα αποτελέσματα του εργαλείου nvprof, βλέπουμε πως τον περισσότερο χρόνο ξοδεύουν οι μεταφορές δεδομένων από και προς το device. Στις υλοποιήσεις μας έως τώρα, μετακινούμε ολόκληρη την εικόνα, με την blocking εντολή cudaMemcpy, μία φορά στην αρχή του προγράμματος από το host στο device και ξανά πίσω όταν ολοκληρωθεί η επεξεργασία. Ωστόσο, ένα kernel block δεν χρησιμοποιεί ολόκληρη την εικόνα. Για τον λόγο αυτό, μόλις τα δεδομένα που αντιστοιχούν στο tile επεξεργασίας ενός kernel block φτάσουν στη συσκευή, μπορούμε άμεσα να ξεκινήσουμε την εκτέλεση, χωρίς να περιμένουμε τη μεταφορά ολόκληρης της εικόνας.

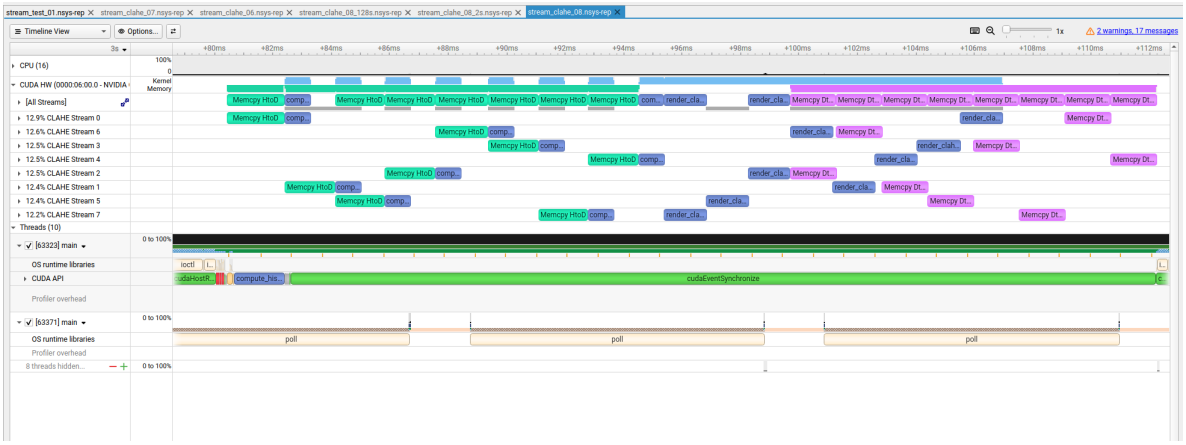
Για την υλοποίηση αυτής της λειτουργίας χρησιμοποιήσαμε Streams. Σε μια αρχική εκδοχή, αντιστοιχούσαμε κάθε tile σε ένα ξεχωριστό stream (one-to-one mapping). Αυτή η προσέγγιση αποδείχθηκε ιδιαίτερα αργή, καθώς απαιτούσε τη δημιουργία πολύ μεγάλου αριθμού streams, επιβαρύνοντας σημαντικά το σύστημα.

Στην τωρινή υλοποίηση ο αριθμός των streams είναι μεταβλητός και καθορίζεται από την παράμετρο `NUM_STREAMS`. Με αυτόν τον τρόπο επιτυγχάνουμε καλύτερη εξισορρόπηση μεταξύ παραλληλισμού και κόστους διαχείρισης των streams. Μετά από πειραματισμό, καταλήξαμε ότι η βέλτιστη τιμή για `NUM_STREAMS` είναι ~ 8.

Η συγκεκριμένη μέθοδος διαχείρισης δεδομένων όμως, επιβάλει πολλαπλές κλίσεις του Kernel για την παραλληλοποίηση των δεδομένων εισόδου, συνεπώς πρέπει να αλλάξουμε το Grid στο οποίο δουλεύει κάθε kernel μεσα σε ένα stream. Πλέον κάνουμε partition τα δεδομένα μας με τρόπο που φαίνεται στην εικόνα 5.



Σχήμα 5: Δομή Streams πάνω στην εικόνα



Σχήμα 6: Μεταφορές H2D (πράσινο), εκτελέσεις kernels (μπλε) και μεταφορές D2H (ροζ).

Χρησιμοποιώντας το εργαλείο nsight systems παρατηρούμε την παραλληλοποίηση μεταξύ των μεταφορών από την GPU στην κύρια μνήμη του CPU και των kernel invocations.

9 Χρήση events

Ανάμεσα από το πρώτο και το δεύτερο βήμα του αλγορίθμου, αντί να χρησιμοποιήσουμε την συνάρτηση `cudaDeviceSynchronize` και να περιμένουμε όλα τα kernel invocations να ολοκληρωθούν, μπορούμε να χρησιμοποιήσουμε events ώστε να ενημερώσουμε το επόμενο kernel πως τα dependencies του είναι έτοιμα. Δηλαδή, από το πρώτο kernel στο δεύτερο, χρειάζεται να έχουν ολοκληρωθεί το προηγούμενο, το τρέχον και το επόμενο stream. Αυτή η έκδοση του αλγορίθμου (έκδοση 9) είναι και η τελική με το μεγαλύτερο throughput.

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU	78.69%	195.07 ms	1	195.07 ms	195.07 ms	195.07 ms	[CUDA memcpy DtoH]
GPU	11.34%	28.100 ms	1	28.100 ms	28.100 ms	28.100 ms	[CUDA memcpy HtoD]
GPU	5.03%	12.473 ms	1	12.473 ms	12.473 ms	12.473 ms	compute_histogram
GPU	4.94%	12.243 ms	1	12.243 ms	12.243 ms	12.243 ms	render_clahe
API	57.23%	224.23 ms	2	112.11 ms	28.153 ms	196.07 ms	cudaMemcpy
API	22.48%	88.079 ms	2	44.039 ms	640 ns	88.078 ms	cudaEventCreate
API	10.35%	40.559 ms	1	40.559 ms	40.559 ms	40.559 ms	cudaDeviceReset
API	6.33%	24.811 ms	9	2.7567 ms	920 ns	12.475 ms	cudaDeviceSynchronize
API	3.37%	13.199 ms	3	4.3997 ms	369.28 μs	6.5288 ms	cudaFree
API	0.09%	358.53 μs	3	119.51 μs	90.306 μs	146.19 μs	cudaMalloc
API	0.08%	297.54 μs	2	148.77 μs	6.6900 μs	290.85 μs	cudaLaunchKernel

Πίνακας 5: Αποτελέσματα του nvprof εκτέλεσης χωρίς streams σε κάρτα γραφικών nvidia GTX 1070.

Run	Average Throughput (MPixels/s)	Std. Deviation	Speed-up vs Previous
streams	1234.382	53.577	-
stream-both	1223.278	70.379	0.991
stream-events	1333.821	55.206	1.091

Table 6: Speed comparison: streams → stream-both → stream-events

10 Δεύτερο kernel

Στις υλοποιήσεις μέχρι στιγμής δεν έχουμε αναφερθεί στον δεύτερο kernel. Για αυτόν μπορούμε να εφαρμόσουμε ένα optimization το οποίο βασίζεται στη μεταφορά των δεδομένων πιθανότητας LUT των γειτονικών tiles στο shared memory. Με αυτόν τον τρόπο, όταν ένα pixel χρειάζεται να υπολογίσει το value του, μπορεί να το αναζητήσει απευθείας στο shared memory, αποφεύγοντας προσβάσεις στο global memory.

Με έναν απλό υπολογισμό βλέπουμε ότι για να αποθηκεύσουμε τα LUT 9 γειτονικών tiles απαιτούνται

$$256 \cdot 4 \cdot 9 = 9216 \text{ bytes.}$$

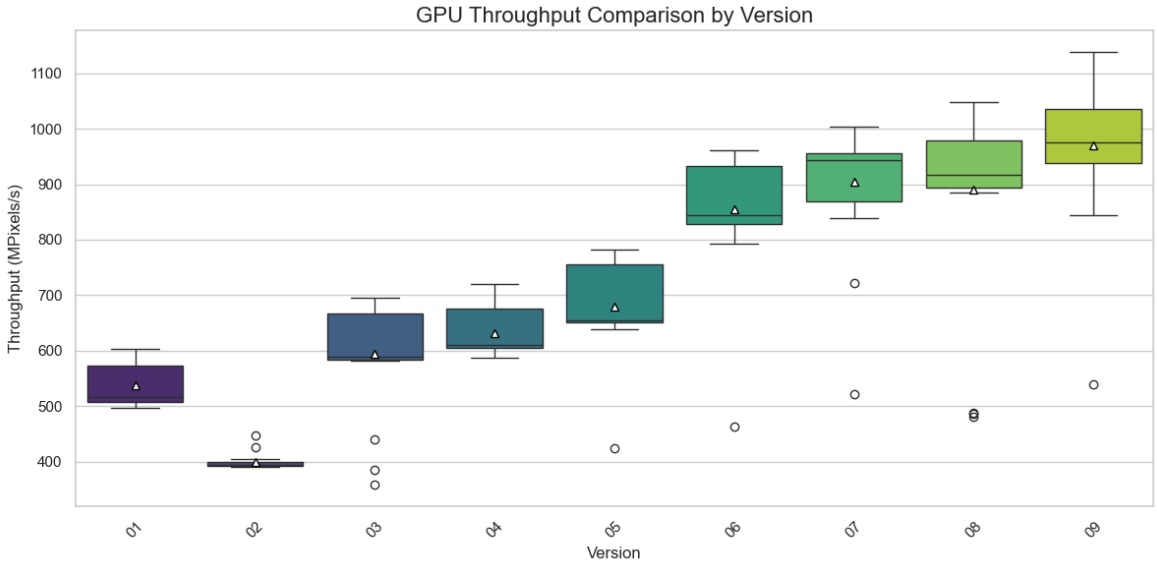
Στην πράξη, όμως, μετά τις αλλαγές που χρειάστηκαν για τον υπολογισμό των νέων indices στον πίνακα shared_lut$[256 \cdot 4 \cdot 9]$, δεν παρατηρήθηκε καμία ουσιαστική βελτίωση στην ταχύτητα. Για αυτό στο τελικό κώδικα δεν έχουμε αλλαγές στην λογική του kernel 2.

11 Χρόνοι εκτέλεσης

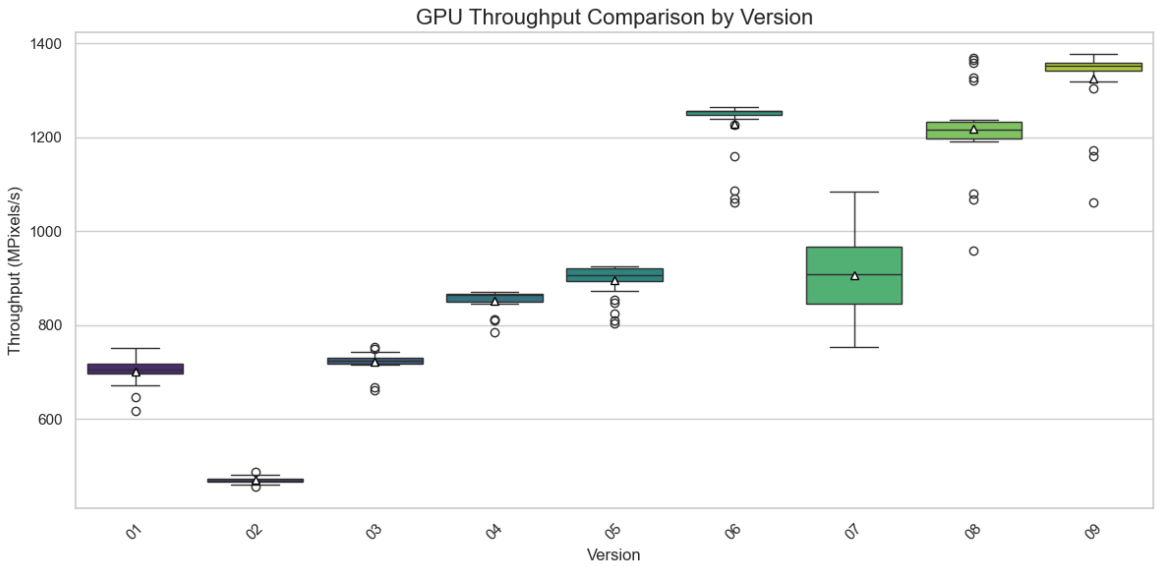
Για την σύγκριση του throughput ανά έκδοση κώδικα, χρησιμοποιήσαμε ένα device από μία κάρτα γραφικών nvidia K80 εκτελώντας την κάθε έκδοση 25 φορές και βγάζοντας τις δύο ακραίες τιμές. Η κάθε έκδοση κώδικα χτίζεται πάνω στην προηγούμενη.

Αριθμός	**Όνομα**
01	3-kernels
02	2-kernels
03	256-threads per tile
04	privatization
05	inclusive parallel scan
06	streams H2D
07	streams D2H
08	streams both
09	events

Πίνακας 7: Εκδόσεις κώδικα.



Σχήμα 7: Throughput της εικόνας `planet_surface.pgm`.



Σχήμα 8: Throughput της εικόνας `senator.pgm`.

Run	Average Throughput (MPixels/s)	Std. Deviation	Speed-up vs 3-kernels
3-kernels	703.491	21.390	1.000
stream-events	1333.821	55.206	1.897
CPU	55.778	3.656	0.079

Πίνακας 8: Σύγκριση αρχικής υλοποίησης και γρηγορότερης υλοποίησης στην GPU με την CPU έκδοση εικόνα `senator.pgm`.